



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

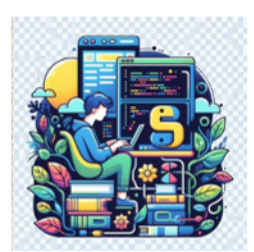
Computer Applications

PROGRAMMING WITH PYTHON

Python Basics and Standard Library Standard Library

Lekha A

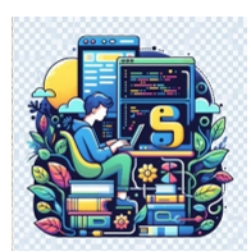
Computer Applications



PROGRAMMING WITH PYTHON

Standard Library

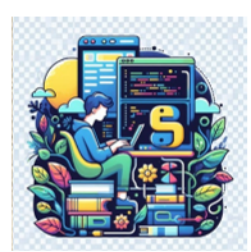
- The Python standard library is a set of modules included with every Python installation.
- A Python library is a collection of related modules.
- It contains bundles of code that can be used repeatedly in different programs.
- It makes Python Programming simpler and convenient for the programmer.



PROGRAMMING WITH PYTHON

Standard Library

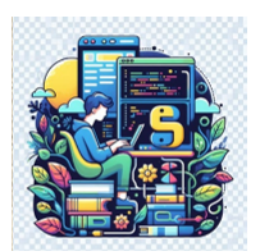
- It is a collection of readily available modules and packages covering various functionalities.
- Written in both C and Python, offering both efficiency and flexibility.
- Distributed with every Python installation, ensuring consistent access across the ecosystem.



PROGRAMMING WITH PYTHON

Standard Library

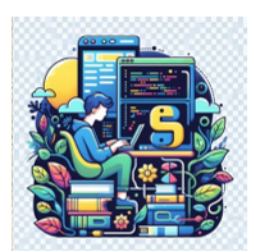
- All these work together to make Python a high-level programming language.
- Python Standard Library plays a very important role.
- Without it, the programmers can't have access to the functionalities of Python.



PROGRAMMING WITH PYTHON

Standard Library

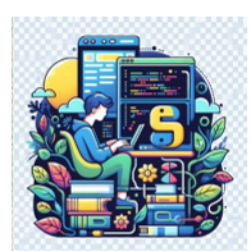
- It offers
 - Essential functionalities
 - Modules for file handling, data structures, networking, threading, math, regular expressions, and more.
 - Web development tools
 - Tools for building web applications, including HTTP requests, JSON processing, templating engines, and scraping libraries.



PROGRAMMING WITH PYTHON

Standard Library

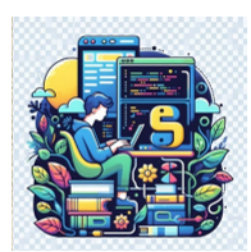
- Databases and data processing
 - Support for SQLite databases, CSV files, and data manipulation tools like Pandas.
- Operating system interaction
 - Access to file systems, processes, and other system resources.
- Testing and debugging
 - Tools for unit testing, code coverage analysis, and logging.



PROGRAMMING WITH PYTHON

Standard Library

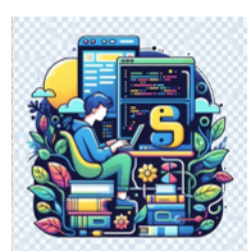
- Benefits of using it
 - Saves time and effort
 - No need to rewrite common functions or modules, accelerating development.
 - Boosts code quality
 - Well-tested and maintained libraries ensure reliable and efficient solutions.



PROGRAMMING WITH PYTHON

Standard Library

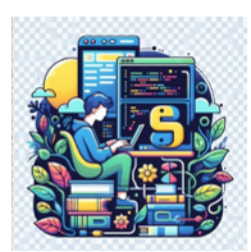
- Improves code readability
 - Standardized modules make code easier to understand and maintain.
- Reduces dependencies
 - Fewer external libraries needed, streamlining project deployment and compatibility.



PROGRAMMING WITH PYTHON

Standard Library

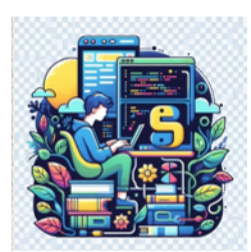
- Some essential modules in the standard library include
 - **os** for interacting with the operating system.
 - **datetime** for working with dates and times.
 - **json** for handling JSON data.
 - **math** for mathematical operations.
 - **re** for regular expressions



PROGRAMMING WITH PYTHON

Standard Library

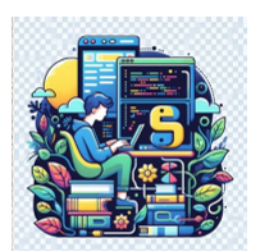
- Some essential modules in the standard library include
 - **collections** for additional data structures.
 - **socket** for networking.
 - **http** for HTTP-related functionality.
 - **sqlite3** for working with SQLite databases.



PROGRAMMING WITH PYTHON

Standard Library - os

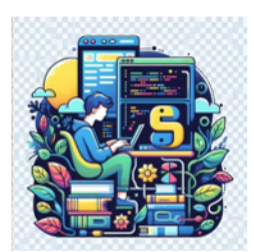
- The os module in Python is a gateway to interacting with the underlying operating system.
- It provides a powerful set of tools for performing various tasks related to file systems, processes, environment variables, and more.



PROGRAMMING WITH PYTHON

Standard Library - os

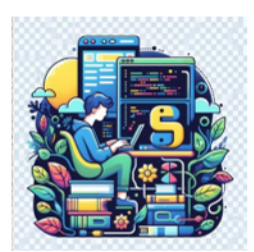
- File System Manipulation
 - Creating, deleting, and moving files and directories
 - Use functions like `os.path.join`, `os.mkdir`, `os.rmdir`, and `os.rename` to manage the file system with ease.



PROGRAMMING WITH PYTHON

Standard Library - os

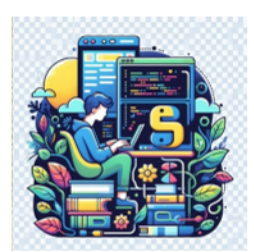
- Listing directory contents
 - Access details about files and directories using functions like `os.listdir` and `os.stat`.
- Reading and writing files
 - Leverage `os.open`, `os.read`, and `os.write` to access file content directly



PROGRAMMING WITH PYTHON

Standard Library - os

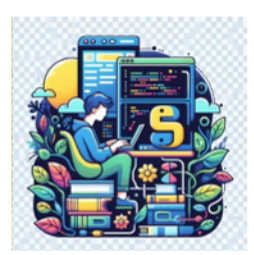
- Process Management
 - Launching new processes
 - Execute external programs through `os.spawnl` or `os.popen` for complex tasks.
 - Managing child processes
 - Monitor and control child processes with functions like `os.wait` and `os.kill`.



PROGRAMMING WITH PYTHON

Standard Library - os

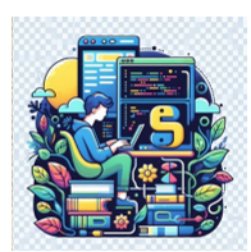
- Process Management
 - Getting process information
 - Use `os.getpid`, `os.getppid`, and `os.environ` to access process IDs and environment variables.



PROGRAMMING WITH PYTHON

Standard Library - os

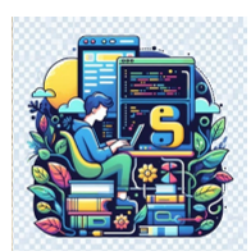
- Getting system information
 - Retrieve details like platform, hostname, and current working directory with `os.name`, `os.hostname`, and `os.getcwd`.
- Manipulating environment variables
 - Access and modify environment variables using `os.environ` and related functions.



PROGRAMMING WITH PYTHON

Standard Library - os

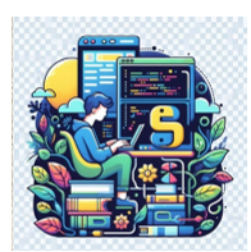
- Error handling
 - Robust error handling mechanisms make os safe and reliable for interacting with the system.



PROGRAMMING WITH PYTHON

Standard Library – os - Benefits of using the os module

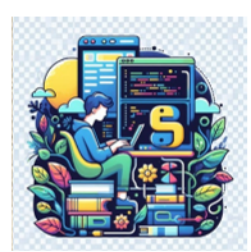
- Cross-platform compatibility
 - Works seamlessly across Windows, macOS, and Linux with platform-specific adjustments.
- Powerful and versatile
 - Offers a wide range of functionalities for interacting with the operating system.



PROGRAMMING WITH PYTHON

Standard Library – os - Benefits of using the os module

- Improves code efficiency
 - Avoids the need for platform-specific libraries for basic system tasks.
- Enhances automation
 - Streamline repetitive tasks involving file manipulation and process management.



PROGRAMMING WITH PYTHON

Standard Library – os - Example

- os - interacting with the operating system

```
import os
```

Get the current working directory

```
current_directory = os.getcwd()  
print("Current working directory:", current_directory)
```

Current working directory: E:\MCA\Notes_MCA\FirstYear\1Sem\Computer Applications\Python\1Sem_2024\Programs\Unit3

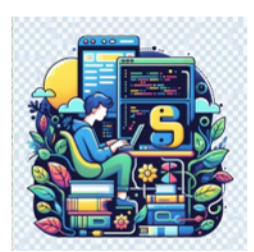
Create a new directory

```
new_dir_name = 'new_directory'  
os.mkdir(new_dir_name)  
if os.path.exists(new_dir_name):  
    print(f"Directory '{new_dir_name}' created successfully!")
```

Directory 'new_directory' created successfully!

File folder

.ipynb_checkpoints	23-01-2025 08:13 PM	File folder
Library	11-01-2025 05:15 AM	File folder
Library Management	11-01-2025 10:00 PM	File folder
Retail Management	10-01-2025 12:05 PM	File folder
Student Management	23-01-2025 08:27 PM	File folder
new_directory	24-01-2025 11:53 AM	File folder



PROGRAMMING WITH PYTHON

Standard Library – os - Example

Change the current working directory

```
new_directory = 'Student Management'  
os.chdir(new_directory)  
updated_directory = os.getcwd()  
print("Updated working directory:", updated_directory)
```

Updated working directory: E:\MCA\Notes_MCA\FirstYear\1Sem\Computer Applications\Python\1Sem_2024\Programs\Unit3\Student Management

Get a list of files and directories in a given path

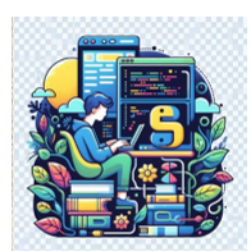
```
files_and_dirs = os.listdir(updated_directory)  
print("Files and directories in the updated directory:", files_and_dirs)
```

Files and directories in the updated directory: ['.ipynb_checkpoints', 'file3.txt', 'file4.txt', 'main.py', 'Student', 'Teacher', 'utilities']

Remove a directory ¶

```
dir_to_remove = 'new_directory'  
if os.path.exists(dir_to_remove):  
    os.rmdir(dir_to_remove)  
    print(f"Directory '{dir_to_remove}' removed successfully!")  
else:  
    print(f"Directory '{dir_to_remove}' does not exist.")
```

Directory 'new_directory' does not exist.



PROGRAMMING WITH PYTHON

Standard Library – os - Example

- Joining path components intelligently

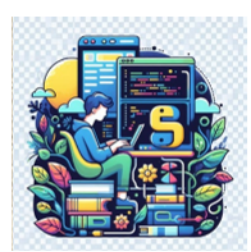
```
path1 = 'E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/'  
path2 = 'Python/1Sem_2024/Programs/Unit3'  
joined_path = os.path.join(path1, path2)  
print("Joined path:", joined_path)
```

Joined path: E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2024/Programs/Unit3

Check if a path exists

```
if os.path.exists(joined_path):  
    print(f"Path '{joined_path}' exists.")  
else:  
    print(f"Path '{joined_path}' does not exist.")
```

Path 'E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2024/Programs/Unit3' exists.



PROGRAMMING WITH PYTHON

Standard Library – os - Example

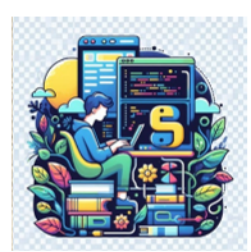
```
path1 = 'D:/E-Books/'  
path2 = 'Big Data'  
joined_path = os.path.join(path1, path2)  
if os.path.exists(joined_path):  
    print(f"Path '{joined_path}' exists.")  
else:  
    print(f"Path '{joined_path}' does not exist.")
```

Path 'D:/E-Books/Big Data' does not exist.

Get the parent process ID (PID) of the current process.

```
try:  
    parent_pid = os.getppid()  
    print("Parent PID:", parent_pid)  
except AttributeError as e:  
    print("AttributeError:", e)
```

Parent PID: 5764



PROGRAMMING WITH PYTHON

Standard Library – os - Example

• Check if a path points to a file

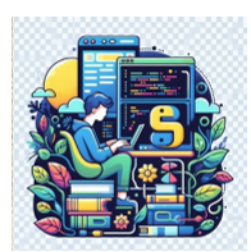
```
file_path = 'D:/E-Books/Technical/Big Data/Programming Hive.pdf'
if os.path.isfile(file_path):
    print(f'{file_path} is a file.')
else:
    print(f'{file_path} is not a file or does not exist.')
```

'D:/E-Books/Technical/Big Data/Programming Hive.pdf' is not a file or does not exist.

Check if a path points to a directory

```
directory_path = 'E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2024/Programs/Unit3'
if os.path.isdir(directory_path):
    print(f'{directory_path} is a directory.')
else:
    print(f'{directory_path} is not a directory or does not exist.')
```

'E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2024/Programs/Unit3' is a directory.



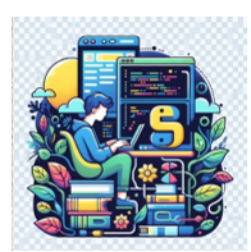
PROGRAMMING WITH PYTHON

Standard Library – os - Example

- Get the file statistics

```
file_path = "E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2024/Programs/Unit3/file3.txt"
file_stats = os.stat(file_path)
print("File Size in Bytes:", file_stats.st_size)
print("File Size in Kilobytes:", file_stats.st_size / 1024)
print("File Size in Megabytes:", file_stats.st_size / (1024 * 1024))
print("File Creation Time:", file_stats.st_ctime)
print("File Modification Time:", file_stats.st_mtime)
print("File Access Time:", file_stats.st_atime)
```

```
File Size in Bytes: 37
File Size in Kilobytes: 0.0361328125
File Size in Megabytes: 3.528594970703125e-05
File Creation Time: 1733731412.922436
File Modification Time: 1734061244.9201846
File Access Time: 1735982349.1485472
```



PROGRAMMING WITH PYTHON

Standard Library – os - Example

- Get the host name

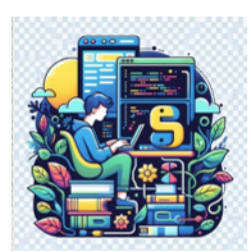
```
import platform
hostname = platform.node()
print(f"Hostname: {hostname}")
```

Hostname: DESKTOP-171DIQB

Get the current user's home directory from the environment variable

```
if "USERPROFILE" in os.environ:
    home_dir = os.environ["USERPROFILE"]
    print(f"Your home directory is: {home_dir}")
else:
    print("Environment variable 'USERPROFILE' does not exist!")
```

Your home directory is: C:\Users\dell



PROGRAMMING WITH PYTHON

Standard Library – os - Example

- Check if a specific API key environment variable is set

```
api_key = os.getenv("API_KEY")

if not api_key:
    print("Error: Missing API key in environment!")
    exit(1)
else:
    print("API key found:", api_key)
```

API key found: api1

Unix Systems

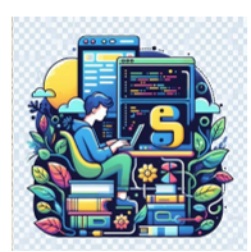
```
result = os.system("ping -c 3 google.com")
if result == 0:
    print("Ping successful!")
else:
    print("Ping failed!")
```

Ping failed!

Windows System

```
result = os.system("ping -n 3 google.com")
if result == 0:
    print("Ping successful!")
else:
    print("Ping failed!")
```

Ping successful!



PROGRAMMING WITH PYTHON

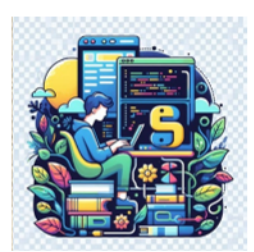
Standard Library – os - Example

- Run a command and capture its output using a pipe

Unix Systems

```
process = os.popen("ls -l")  
output = process.read()  
print(f"Process output:\n{output}")  
process.close()
```

Process output:



PROGRAMMING WITH PYTHON

Standard Library – os - Example

- Windows System

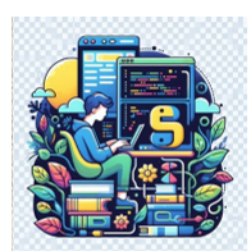
```
process = os.popen("dir")
output = process.read()
print(f"Process output:\n{output}")
process.close()
```

Process output:

Volume in drive E is New Volume
Volume Serial Number is FEB5-CD41

Directory of E:\MCA\Notes_MCA\FirstYear\1Sem\Computer Applications\Python\1Sem_2024\Programs\Unit3\Student Management

23-01-2025	08:27 PM	<DIR>	.
24-01-2025	12:12 PM	<DIR>	..
10-01-2025	12:26 PM	<DIR>	.ipynb_checkpoints
23-01-2025	08:24 PM		740 file3.txt
23-01-2025	08:27 PM		740 file4.txt
10-01-2025	12:28 PM		272 main.py
10-01-2025	12:27 PM	<DIR>	Student
10-01-2025	12:27 PM	<DIR>	Teacher
10-01-2025	12:27 PM	<DIR>	utilities
		3 File(s)	1,752 bytes
		6 Dir(s)	71,787,933,696 bytes free



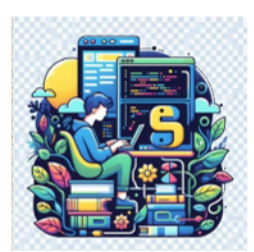
PROGRAMMING WITH PYTHON

Standard Library – os - Example

- Dictionary-like object containing the current environment variables

```
environment_variables = os.environ
print("Environment Variables:")
for key, value in environment_variables.items():
    print(f"{key}: {value}")
```

```
Environment Variables:
ALLUSERSPROFILE: C:\ProgramData
API_KEY: api1
APPDATA: C:\Users\dell\AppData\Roaming
ASL.LOG: Destination=file
COMMONPROGRAMFILES: C:\Program Files\Common Files
COMMONPROGRAMFILES(X86): C:\Program Files (x86)\Common Files
COMMONPROGRAMW6432: C:\Program Files\Common Files
COMPUTERNAME: DESKTOP-171DIQB
COMSPEC: C:\WINDOWS\system32\cmd.exe
DRIVERDATA: C:\Windows\System32\Drivers\DriverData
```



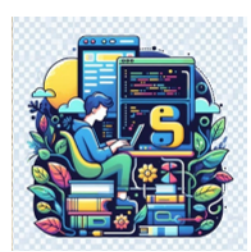
PROGRAMMING WITH PYTHON

Standard Library – os - Example

Get the value of an environment variable

```
specific_variable = 'PATH'  
var_value = os.getenv(specific_variable)  
print(f"\nValue of '{specific_variable}' variable: {var_value}")
```

Value of 'PATH' variable: C:\Program Files\Common Files\Oracle\Java\javapath;C:\Program Files (x86)\Common Files\Oracle\Java\javapath;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShell\v1.0\;C:\WINDOWS\System32\OpenSSH\;C:\Program Files\dotnet\;C:\TDM-GCC-64\bin;C:\Users\dell\AppData\Local\Programs\Python\Python311;C:\Program Files\gnuplot\bin;C:\Program Files\Git\cmd;C:\Java\bin;C:\hadoop\bin;C:\hadoop\sbin;C:\Program Files\Calibre2\;C:\Program Files\MySQL\MySQL Shell 8.0\bin\;C:\Users\dell\AppData\Local\Programs\Python\Python312\Scripts\;C:\Users\dell\AppData\Local\Programs\Python\Python312\;C:\Users\dell\AppData\Local\Microsoft\WindowsApps;C:\Users\dell\AppData\Local\Programs\Microsoft VS Code\bin;C:\TDM-GCC-64\bin;C:\Users\dell\AppData\Local\Programs\Python\Python311;C:\Program Files\gnuplot\bin;C:\Users\dell\AppData\Local\Programs\Python\Python311\Scripts\jupyter-notebook.exe;C:\java\bin;C:\hadoop\bin\bin;C:\hadoop\bin;C:\hadoop\sbin;



PROGRAMMING WITH PYTHON

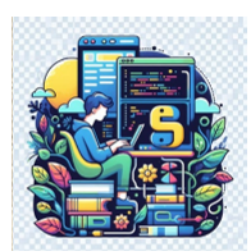
Standard Library – os - Example

```
command_to_pipe = "dir E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2023/Programs/Unit 3/ | file3.txt"
pipe = os.popen(command_to_pipe)
output = pipe.read()
print("\nOutput of the piped command:")
print(output)
```

Output of the piped command:

```
file3.txt
File Edit View

.ipynb_checkpoints
books.xml
Books_dom.xml
books_soup.xml
data.csv
data1.csv
dataframe.pkl
data_2.csv
dict.pkl
empty.txt
empty_file.txt
emp_det.csv
emp_det.txt
Errors.html
Errors.ipynb
file1.txt
file2.txt
file3.txt
FileHandlingText.ipynb
FilesCSV.ipynb
FilesJSON.ipynb
FilesXML.ipynb
greet_function.pkl
Library
Library Management
list.pkl
multiple.pkl
myfile.xml
names.csv
new_directory
output.csv
output_1.csv
```



PROGRAMMING WITH PYTHON

Standard Library – os - Example

```
• directory_path = 'E:/MCA/Notes_MCA/FirstYear/1Sem/Computer Applications/Python/1Sem_2024/Programs/Unit3'
if os.path.exists(directory_path):
    files = os.listdir(directory_path)
    if files:
        output_file_path = 'file4.txt'
        with open(output_file_path, 'w') as file:
            for filename in files:
                file.write(f"{filename}\n")
        print(f"Directory contents have been written to '{output_file_path}'.")
    else:
        print(f"The directory '{directory_path}' is empty. No data written to 'file3.txt'.")
else:
    print(f"The directory '{directory_path}' does not exist. Please check the path and try again.")
```

Directory contents have been written to 'file4.txt'.



PROGRAMMING WITH PYTHON

Recap

- Standard Library
- OS
 - File System Manipulation
 - Process Management
 - Getting system information
 - Manipulating environment variables
 - Error handling



THANK YOU

Lekha A
Computer Applications